Z **ZoonoseApp** Scaffold v1.0

Expo + Supabase + Next.js · Android-first MVP

1 Estrutura de Diretórios

Monorepo com /app (Expo), /web (Next.js), /supabase e /scripts

📁 zoonoseapp/ — monorepo root

```
zoonoseapp/
├── app/                          # Expo React Native
│   ├── src/
│   │   ├── screens/
│   │   │   ├── LoginScreen.tsx
│   │   │   ├── HomeScreen.tsx
│   │   │   ├── PropertyDetailScreen.tsx
│   │   │   ├── RegisterVisitScreen.tsx
│   │   │   ├── ConfirmVisitScreen.tsx
│   │   │   ├── HistoryScreen.tsx
│   │   │   ├── SyncStatusScreen.tsx
│   │   │   └── SettingsScreen.tsx
│   │   ├── hooks/
│   │   │   ├── useOfflineSync.ts
│   │   │   ├── useAuth.ts
│   │   │   └── useProperties.ts
│   │   ├── lib/
│   │   │   ├── supabase.ts
│   │   │   └── mmkv.ts
│   │   ├── types/index.ts
│   │   └── navigation/AppNavigator.tsx
│   ├── app.json
│   ├── eas.json
│   └── package.json
├── web/                          # Next.js — painel coordenador
│   ├── app/
│   │   ├── (auth)/login/page.tsx
│   │   ├── dashboard/page.tsx
│   │   ├── dashboard/imoveis/page.tsx
│   │   ├── dashboard/imoveis/[id]/page.tsx
│   │   ├── dashboard/agentes/page.tsx
│   │   └── api/export/route.ts
│   ├── components/
│   │   ├── MetricCard.tsx
```

Z **ZoonoseApp** Scaffold v1.0 Expo + Supabase + Next.js · Android-first MVP

```
│   └── package.json
├── supabase/
│   ├── migrations/
│   │   ├── 001_schema.sql
│   │   ├── 002_indexes.sql
│   │   └── 003_seed.sql
│   └── config.toml
└── scripts/
    └── import-imoveis.js
```

2 Banco de Dados — Supabase Schema

SQL completo para o Supabase SQL Editor: 6 tabelas, RLS policies, índices.

supabase/migrations/001_schema.sql

```sql
-- ZOONOSEAPP MVP -- SCHEMA COMPLETO

CREATE TABLE sectors (
  id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name         TEXT NOT NULL,
  municipality TEXT NOT NULL DEFAULT 'Uberlandia',
  created_at   TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE profiles (
  id         UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  name       TEXT NOT NULL,
  role       TEXT NOT NULL CHECK (role IN ('agent','coordinator','manager','admin')),
  sector_id  UUID REFERENCES sectors(id),
  active     BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE OR REPLACE FUNCTION handle_new_user() RETURNS TRIGGER AS $$
BEGIN
  INSERT INTO profiles (id, name, role) VALUES
  (NEW.id, NEW.raw_user_meta_data->>'name', COALESCE(NEW.raw_user_meta_data->>'role',
```

ZoonoseApp Scaffold v1.0 Expo + Supabase + Next.js · Android-first MVP

```
CREATE TRIGGER on_auth_user_created
  AFTER INSERT ON auth.users
  FOR EACH ROW EXECUTE FUNCTION handle_new_user();

CREATE TABLE properties (
  id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  code       TEXT UNIQUE NOT NULL,
  address    TEXT NOT NULL,
  sector_id  UUID REFERENCES sectors(id),
  type       TEXT DEFAULT 'residencial'
               CHECK (type IN ('residencial','comercial','terreno','especial')),
  lat        DOUBLE PRECISION,
  lng        DOUBLE PRECISION,
  active     BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE visits (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  property_id UUID NOT NULL REFERENCES properties(id),
  agent_id    UUID NOT NULL REFERENCES profiles(id),
  status      TEXT NOT NULL CHECK (status IN (
                'visitado_sem_foco','visitado_com_achado','fechado',
                'recusado','nao_localizado','pendente_revisao')),
  notes       TEXT,
  visited_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  synced_at   TIMESTAMPTZ,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE evidences (
  id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  visit_id     UUID NOT NULL REFERENCES visits(id) ON DELETE CASCADE,
  storage_path TEXT NOT NULL,
  created_at   TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE activity_logs (
  id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id    UUID REFERENCES profiles(id),
  action     TEXT NOT NULL,
  entity     TEXT, entity_id UUID,
  created_at TIMESTAMPTZ DEFAULT NOW()
```

ZoonoseApp Scaffold v1.0 Expo + Supabase + Next.js · Android-first MVP

```
ALTER TABLE visits     ENABLE ROW LEVEL SECURITY;
ALTER TABLE profiles   ENABLE ROW LEVEL SECURITY;
ALTER TABLE properties ENABLE ROW LEVEL SECURITY;
ALTER TABLE evidences  ENABLE ROW LEVEL SECURITY;

CREATE POLICY "profiles_self" ON profiles FOR SELECT USING
  (id = auth.uid() OR EXISTS (SELECT 1 FROM profiles p
   WHERE p.id = auth.uid() AND p.role IN ('admin','manager')));

CREATE POLICY "properties_read" ON properties FOR SELECT USING (auth.role() =
'authenticated');

CREATE POLICY "visits_read" ON visits FOR SELECT USING (
  agent_id = auth.uid() OR EXISTS (SELECT 1 FROM profiles p
  WHERE p.id = auth.uid() AND p.role IN ('coordinator','manager','admin')));

CREATE POLICY "visits_insert" ON visits FOR INSERT WITH CHECK (agent_id = auth.uid());

-- Indices
CREATE INDEX idx_visits_property   ON visits(property_id);
CREATE INDEX idx_visits_agent      ON visits(agent_id);
CREATE INDEX idx_visits_status     ON visits(status);
CREATE INDEX idx_visits_visited_at ON visits(visited_at DESC);
CREATE INDEX idx_properties_code   ON properties(code);
CREATE INDEX idx_properties_sector ON properties(sector_id);
```

supabase/migrations/003_seed.sql

```
-- SEED DE DEMO
INSERT INTO sectors (id, name) VALUES
  ('11111111-0000-0000-0000-000000000001', 'Setor Norte'),
  ('11111111-0000-0000-0000-000000000002', 'Setor Sul');

INSERT INTO properties (code, address, sector_id, type) VALUES
  ('UBL-0001','Rua das Acacias, 123 Tibery', '11111111-0000-0000-0000-
000000000001','residencial'),
  ('UBL-0002','Av Joao Naves de Avila, 456', '11111111-0000-0000-0000-
000000000001','residencial'),
  ('UBL-0003','Rua Olegario Maciel, 78 Centro', '11111111-0000-0000-0000-
000000000002','comercial'),
  ('UBL-0004','Rua Tupinambas, 22 Roosevelt', '11111111-0000-0000-0000-
```

```
  ('UBL-0006','Rua Goias, 510 Centro', '11111111-0000-0000-0000-
000000000002','comercial'),
  ('UBL-0007','Rua Ceara, 33 Bairro Cazeca', '11111111-0000-0000-0000-
000000000001','residencial'),
  ('UBL-0008','Rua Para, 88 Osvaldo Rezende', '11111111-0000-0000-0000-
000000000001','residencial'),
  ('UBL-0009','Av Afonso Pena, 1200 Karaiba', '11111111-0000-0000-0000-
000000000002','residencial'),
  ('UBL-0010','Rua Espirito Santo, 55 Bairro Brasil','11111111-0000-0000-0000-
000000000001','terreno');
```

3 App Mobile — Expo React Native

Código completo das telas do agente de campo. TypeScript + Supabase + MMKV offline.

📦 app/package.json

```
{
  "name": "zoonoseapp",  "version": "1.0.0",
  "main": "expo-router/entry",
  "scripts": {
    "start": "expo start",
    "android": "expo run:android",
    "build:apk": "eas build --platform android --profile preview"
  },
  "dependencies": {
    "expo": "~51.0.0",
    "@supabase/supabase-js": "^2.43.0",
    "react-native-mmkv": "^2.12.0",
    "expo-image-picker": "~15.0.7",
    "expo-camera": "~15.0.15",
    "@react-native-community/netinfo": "11.3.1",
    "@react-navigation/native": "^6.1.17",
    "@react-navigation/native-stack": "^6.9.26",
    "react-native-safe-area-context": "4.10.1",
    "react-native-screens": "3.31.1"
```

ZoonoseApp Scaffold v1.0 Expo + Supabase + Next.js · Android-first MVP

app/src/lib/supabase.ts

```
import { createClient } from '@supabase/supabase-js'
import { MMKV } from 'react-native-mmkv'

const storage = new MMKV()
const supabaseMMKVAdapter = {
  getItem:    (key: string) => storage.getString(key) ?? null,
  setItem:    (key: string, value: string) => storage.set(key, value),
  removeItem: (key: string) => storage.delete(key),
}

export const supabase = createClient(
  process.env.EXPO_PUBLIC_SUPABASE_URL!,
  process.env.EXPO_PUBLIC_SUPABASE_ANON_KEY!,
  { auth: { storage: supabaseMMKVAdapter, autoRefreshToken: true,
      persistSession: true, detectSessionInUrl: false } }
)
```

app/src/lib/mmkv.ts

```
import { MMKV } from 'react-native-mmkv'
export const storage = new MMKV()

export const offlineQueue = {
  getAll: (): PendingVisit[] => {
    const raw = storage.getString('offline_queue')
    return raw ? JSON.parse(raw) : []
  },
  add: (visit: PendingVisit) => {
    const q = offlineQueue.getAll(); q.push(visit)
    storage.set('offline_queue', JSON.stringify(q))
  },
  remove: (id: string) => {
    storage.set('offline_queue',
      JSON.stringify(offlineQueue.getAll().filter(v => v.localId !== id)))
  },
  clear: () => storage.delete('offline_queue'),
}
```

ZoonoseApp Scaffold v1.0 Expo + Supabase + Next.js · Android-first MVP

```
}
```

app/src/types/index.ts

```
export type Role = 'agent' | 'coordinator' | 'manager' | 'admin'

export type VisitStatus =
  | 'visitado_sem_foco' | 'visitado_com_achado' | 'fechado'
  | 'recusado' | 'nao_localizado' | 'pendente_revisao'

export interface Profile {
  id: string; name: string; role: Role
  sector_id: string | null; active: boolean; created_at: string
}

export interface Property {
  id: string; code: string; address: string; sector_id: string
  type: 'residencial' | 'comercial' | 'terreno' | 'especial'
  lat?: number; lng?: number; active: boolean
}

export interface Visit {
  id: string; property_id: string; agent_id: string
  status: VisitStatus; notes?: string
  visited_at: string; synced_at?: string; created_at: string
}

export const STATUS_CONFIG: Record<VisitStatus, { label: string; color: string; bg:
string }> = {
  visitado_sem_foco:   { label: 'Sem Foco',      color: '#10b981', bg:
'rgba(16,185,129,0.15)' },
  visitado_com_achado: { label: 'Com Achado',     color: '#ef4444', bg:
'rgba(239,68,68,0.15)' },
  fechado:             { label: 'Fechado',         color: '#94a3b8', bg:
'rgba(148,163,184,0.15)' },
  recusado:            { label: 'Recusado',        color: '#f97316', bg:
'rgba(249,115,22,0.15)' },
  nao_localizado:      { label: 'Nao Localizado',  color: '#f59e0b', bg:
'rgba(245,158,11,0.15)' },
  pendente_revisao:    { label: 'Pend. Revisao',   color: '#3b82f6', bg:
```

ZoonoseApp Scaffold v1.0 Expo + Supabase + Next.js · Android-first MVP

app/src/hooks/useOfflineSync.ts

```
import { useEffect, useCallback } from 'react'
import NetInfo from '@react-native-community/netinfo'
import { supabase } from '../lib/supabase'
import { offlineQueue } from '../lib/mmkv'

export function useOfflineSync() {
  const syncPending = useCallback(async () => {
    const queue = offlineQueue.getAll()
    if (queue.length === 0) return
    for (const item of queue) {
      try {
        const { error } = await supabase.from('visits').insert({
          ...item, synced_at: new Date().toISOString(),
        })
        if (!error) offlineQueue.remove(item.localId)
      } catch (e) { console.warn('Sync falhou', item.localId, e) }
    }
  }, [])

  useEffect(() => {
    const unsub = NetInfo.addEventListener(s => { if (s.isConnected) syncPending() })
    return unsub
  }, [syncPending])

  return { syncPending }
}
```

app/src/hooks/useAuth.ts

```
import { useState, useEffect } from 'react'
import { supabase } from '../lib/supabase'
import { Profile } from '../types'

export function useAuth() {
  const [profile, setProfile] = useState<Profile | null>(null)
  const [loading, setLoading] = useState(true)

  useEffect(() => {
```

ZoonoseApp Scaffold v1.0 Expo + Supabase + Next.js · Android-first MVP

```
      const { data } = await supabase
        .from('profiles').select('*').eq('id', session.user.id).single()
      setProfile(data)
    } else { setProfile(null) }
    setLoading(false)
  })
  return () => subscription.unsubscribe()
}, [])

const signIn = async (email: string, password: string) => {
  const { error } = await supabase.auth.signInWithPassword({ email, password })
  return { error }
}
const signOut = async () => { await supabase.auth.signOut() }
return { profile, loading, signIn, signOut }
}
```

app/src/navigation/AppNavigator.tsx

```
import React from 'react'
import { NavigationContainer } from '@react-navigation/native'
import { createNativeStackNavigator } from '@react-navigation/native-stack'
import { useAuth } from '../hooks/useAuth'
import LoginScreen from '../screens/LoginScreen'
import HomeScreen from '../screens/HomeScreen'
import PropertyDetailScreen from '../screens/PropertyDetailScreen'
import RegisterVisitScreen from '../screens/RegisterVisitScreen'
import ConfirmVisitScreen from '../screens/ConfirmVisitScreen'
import HistoryScreen from '../screens/HistoryScreen'
import SyncStatusScreen from '../screens/SyncStatusScreen'
import SettingsScreen from '../screens/SettingsScreen'

export type RootStackParamList = {
  Login: undefined; Home: undefined
  PropertyDetail: { propertyId: string }
  RegisterVisit: { propertyId: string; propertyCode: string }
  ConfirmVisit: { visitData: any }
  History: undefined; SyncStatus: undefined; Settings: undefined
}

const Stack = createNativeStackNavigator<RootStackParamList>()
```

ZoonoseApp Scaffold v1.0 Expo + Supabase + Next.js · Android-first MVP

```
  if (loading) return null
  return (
    <NavigationContainer>
      <Stack.Navigator screenOptions={{
          headerStyle: { backgroundColor: '#111d35' },
          headerTintColor: '#e2e8f0',
          contentStyle: { backgroundColor: '#0a1628' } }}>
        {!profile ? (
          <Stack.Screen name="Login" component={LoginScreen} options={{ headerShown:
false }} />
        ) : (
          <>
            <Stack.Screen name="Home" component={HomeScreen} options={{ title: "Buscar
Imovel" }} />
            <Stack.Screen name="PropertyDetail" component={PropertyDetailScreen}
options={{ title: "Imovel" }} />
            <Stack.Screen name="RegisterVisit" component={RegisterVisitScreen} options=
{{ title: "Registrar Visita" }} />
            <Stack.Screen name="ConfirmVisit" component={ConfirmVisitScreen} options={{
title: "Confirmar" }} />
            <Stack.Screen name="History" component={HistoryScreen} options={{ title:
"Meu Historico" }} />
            <Stack.Screen name="SyncStatus" component={SyncStatusScreen} options={{
title: "Sincronizacao" }} />
            <Stack.Screen name="Settings" component={SettingsScreen} options={{ title:
"Configuracoes" }} />
          </>
        )}
      </Stack.Navigator>
    </NavigationContainer>
  )
}
```

📄 app/src/screens/LoginScreen.tsx

```
import React, { useState } from 'react'
import { View, Text, TextInput, TouchableOpacity, ActivityIndicator, StyleSheet, Alert
} from 'react-native'
import { useAuth } from '../hooks/useAuth'

export default function LoginScreen() {
```

ZoonoseApp Scaffold v1.0 Expo + Supabase + Next.js · Android-first MVP

```
  const { signIn } = useAuth()

  const handleLogin = async () => {
    if (!email || !password) { Alert.alert('Erro', 'Preencha e-mail e senha.'); return
}
    setLoading(true)
    const { error } = await signIn(email, password)
    setLoading(false)
    if (error) Alert.alert('Erro de login', error.message)
  }

  return (
    <View style={s.container}>
      <View style={s.card}>
        <View style={s.logo}><Text style={s.logoText}>Z</Text></View>
        <Text style={s.title}>ZoonoseApp</Text>
        <Text style={s.subtitle}>Vigilancia Territorial</Text>
        <TextInput style={s.input} placeholder="E-mail" placeholderTextColor="#64748b"
          value={email} onChangeText={setEmail} keyboardType="email-address"
autoCapitalize="none" />
        <TextInput style={s.input} placeholder="Senha" placeholderTextColor="#64748b"
          value={password} onChangeText={setPassword} secureTextEntry />
        <TouchableOpacity style={s.btn} onPress={handleLogin} disabled={loading}>
          {loading ? <ActivityIndicator color="#fff" /> : <Text style=
{s.btnText}>Entrar</Text>}
        </TouchableOpacity>
      </View>
    </View>
  )
}

const s = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#0a1628', alignItems: 'center',
justifyContent: 'center', padding: 24 },
  card:      { width: '100%', maxWidth: 360, backgroundColor: '#111d35', borderRadius:
16,
               padding: 28, borderWidth: 1, borderColor: '#1e3a5f', alignItems:
'center' },
  logo:      { width: 56, height: 56, borderRadius: 14, backgroundColor: '#3b82f6',
               alignItems: 'center', justifyContent: 'center', marginBottom: 16 },
  logoText:  { color: '#fff', fontWeight: '800', fontSize: 22 },
  title:     { color: '#e2e8f0', fontWeight: '700', fontSize: 22, marginBottom: 4 },
  subtitle:  { color: '#94a3b8', fontSize: 13, marginBottom: 28 },
```

ZoonoseApp Scaffold v1.0 Expo + Supabase + Next.js · Android-first MVP

```
marginBottom: 12 },
  btn:       { width: '100%', backgroundColor: '#3b82f6', borderRadius: 10,
               padding: 16, alignItems: 'center', marginTop: 8 },
  btnText:   { color: '#fff', fontWeight: '700', fontSize: 15 },
})
```

app/src/screens/HomeScreen.tsx

```
import React, { useState, useEffect } from 'react'
import { View, Text, TextInput, FlatList, TouchableOpacity, StyleSheet,
ActivityIndicator } from 'react-native'
import { useNavigation } from '@react-navigation/native'
import { supabase } from '../lib/supabase'
import { offlineQueue } from '../lib/mmkv'
import { Property } from '../types'

export default function HomeScreen() {
  const nav = useNavigation()
  const [query, setQuery] = useState('')
  const [properties, setProperties] = useState<Property[]>([])
  const [loading, setLoading] = useState(false)
  const pendingCount = offlineQueue.getAll().length

  const search = async (q: string) => {
    if (q.length < 2) { setProperties([]); return }
    setLoading(true)
    const { data } = await supabase
      .from('properties').select('*')
      .or(`code.ilike.%${q}%,address.ilike.%${q}%`)
      .eq('active', true).limit(20)
    setProperties(data ?? [])
    setLoading(false)
  }

  useEffect(() => { search(query) }, [query])

  return (
    <View style={s.container}>
      {pendingCount > 0 && (
        <TouchableOpacity style={s.syncBanner}
          onPress={() => nav.navigate('SyncStatus' as any)}>
```

ZoonoseApp Scaffold v1.0 Expo + Supabase + Next.js · Android-first MVP

```
      )}
      <TextInput style={s.search} placeholder="Buscar por codigo ou endereco..."
        placeholderTextColor="#64748b" value={query} onChangeText={setQuery} />
      {loading && <ActivityIndicator color="#3b82f6" style={{ marginTop: 16 }} />}
      <FlatList data={properties} keyExtractor={p => p.id}
        renderItem={({ item }) => (
          <TouchableOpacity style={s.item}
            onPress={() => nav.navigate('PropertyDetail' as any, { propertyId: item.id
})}>
            <Text style={s.itemCode}>{item.code}</Text>
            <Text style={s.itemAddress} numberOfLines={1}>{item.address}</Text>
          </TouchableOpacity>
        )} />
    </View>
  )
}

const s = StyleSheet.create({
  container:      { flex: 1, backgroundColor: '#0a1628', padding: 16 },
  syncBanner:     { backgroundColor: 'rgba(59,130,246,0.15)', borderWidth: 1,
borderColor: '#3b82f6',
                    borderRadius: 10, padding: 12, marginBottom: 12, alignItems:
'center' },
  syncBannerText: { color: '#60a5fa', fontWeight: '600', fontSize: 13 },
  search:         { backgroundColor: '#111d35', borderWidth: 1, borderColor: '#1e3a5f',
                    borderRadius: 10, padding: 14, color: '#e2e8f0', marginBottom: 12
},
  item:           { flexDirection: 'row', backgroundColor: '#111d35', borderRadius: 10,
                    padding: 14, marginBottom: 8, borderWidth: 1, borderColor:
'#1e3a5f', gap: 12 },
  itemCode:       { color: '#60a5fa', fontWeight: '700', fontSize: 12, minWidth: 72 },
  itemAddress:    { color: '#e2e8f0', flex: 1, fontSize: 13 },
})
```

app/src/screens/RegisterVisitScreen.tsx

```
import React, { useState } from 'react'
import { View, Text, TouchableOpacity, TextInput, ScrollView, Alert, ActivityIndicator
} from 'react-native'
import { useRoute, useNavigation } from '@react-navigation/native'
import NetInfo from '@react-native-community/netinfo'
```

ZoonoseApp Scaffold v1.0 Expo + Supabase + Next.js · Android-first MVP

```
import * as ImagePicker from 'expo-image-picker'
import { supabase } from '../lib/supabase'
import { offlineQueue } from '../lib/mmkv'
import { useAuth } from '../hooks/useAuth'
import { VisitStatus, STATUS_CONFIG } from '../types'

export default function RegisterVisitScreen() {
  const route = useRoute() as any
  const nav = useNavigation()
  const { profile } = useAuth()
  const [selectedStatus, setSelectedStatus] = useState<VisitStatus | null>(null)
  const [notes, setNotes] = useState('')
  const [photoUri, setPhotoUri] = useState<string | null>(null)
  const [saving, setSaving] = useState(false)

  const pickPhoto = async () => {
    const result = await ImagePicker.launchCameraAsync({ quality: 0.6 })
    if (!result.canceled) setPhotoUri(result.assets[0].uri)
  }

  const handleSave = async () => {
    if (!selectedStatus) { Alert.alert('Selecione o status'); return }
    if (!profile) return
    setSaving(true)
    const state = await NetInfo.fetch()
    const visitData = {
      property_id: route.params.propertyId,
      agent_id: profile.id, status: selectedStatus,
      notes: notes || undefined,
      visited_at: new Date().toISOString(),
    }
    if (state.isConnected) {
      const { error } = await supabase.from('visits').insert({
        ...visitData, synced_at: new Date().toISOString()
      })
      if (error) { Alert.alert('Erro', error.message); setSaving(false); return }
    } else {
      offlineQueue.add({ ...visitData, localId: Date.now().toString(), photoUri:
photoUri ?? undefined })
    }
    setSaving(false)
    nav.navigate('ConfirmVisit' as any, { visitData: { ...visitData, offline:
!state.isConnected } })
  }

  return (
```

ZoonoseApp Scaffold v1.0 Expo + Supabase + Next.js · Android-first MVP

```
        Imovel: <Text style={{ color: '#60a5fa' }}>{route.params?.propertyCode}</Text>
      </Text>
      <Text style={{ color: '#94a3b8', marginTop: 20, marginBottom: 10, fontWeight:
'700' }}>
        STATUS DA VISITA
      </Text>
      <View style={{ flexDirection: 'row', flexWrap: 'wrap', gap: 8, marginBottom: 20
}}>
        {(Object.entries(STATUS_CONFIG) as [VisitStatus, any][]).map(([key, cfg]) => (
          <TouchableOpacity key={key}
            style={{ borderWidth: 2, borderRadius: 10, paddingHorizontal: 14,
paddingVertical: 10,
              minWidth: '47%', borderColor: cfg.color,
              backgroundColor: selectedStatus === key ? cfg.bg : 'transparent' }}
            onPress={() => setSelectedStatus(key)}>
            <Text style={{ color: cfg.color, fontWeight: '700', textAlign: 'center' }}>
{cfg.label}</Text>
          </TouchableOpacity>
        ))}
      </View>
      <TextInput
        style={{ backgroundColor: '#111d35', borderWidth: 1, borderColor: '#1e3a5f',
          borderRadius: 10, padding: 14, color: '#e2e8f0', marginBottom: 16 }}
        placeholder="Nota opcional..." placeholderTextColor="#64748b"
        value={notes} onChangeText={setNotes} multiline numberOfLines={3} />
      <TouchableOpacity
        style={{ backgroundColor: '#111d35', borderWidth: 1, borderColor: '#1e3a5f',
          borderRadius: 10, padding: 14, alignItems: 'center', marginBottom: 20 }}
        onPress={pickPhoto}>
        <Text style={{ color: '#94a3b8' }}>
          {photoUri ? 'Foto capturada' : 'Tirar foto (opcional)'}
        </Text>
      </TouchableOpacity>
      <TouchableOpacity
        style={{ backgroundColor: '#3b82f6', borderRadius: 10, padding: 16,
          alignItems: 'center', opacity: saving ? 0.6 : 1 }}
        onPress={handleSave} disabled={saving}>
        {saving
          ? <ActivityIndicator color="#fff" />
          : <Text style={{ color: '#fff', fontWeight: '700' }}>Confirmar visita</Text>}
      </TouchableOpacity>
    </ScrollView>
```

4 Painel Web — Next.js + Tailwind

Dashboard do coordenador. Deploy na Vercel. Métricas de cobertura, pendências e reincidências.

📦 web/package.json

```
{
  "name": "zoonoseapp-web", "version": "1.0.0",
  "scripts": { "dev": "next dev", "build": "next build", "start": "next start" },
  "dependencies": {
    "next": "14.2.0",  "react": "^18",
    "@supabase/supabase-js": "^2.43.0",
    "@supabase/ssr": "^0.3.0",
    "tailwindcss": "^3.4.0",
    "autoprefixer": "^10", "postcss": "^8"
  },
  "devDependencies": { "@types/react": "^18", "typescript": "^5" }
}
```

📄 web/lib/supabase.ts

```
import { createBrowserClient } from '@supabase/ssr'

export function createClient() {
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  )
}
```

📄 web/app/dashboard/page.tsx

```
import { createClient } from '@/lib/supabase'
import MetricCard from '@/components/MetricCard'
```

ZoonoseApp Scaffold v1.0 Expo + Supabase + Next.js · Android-first MVP

```
  const today = new Date().toISOString().split('T')[0]

  const [
    { count: total }, { count: visitedToday },
    { count: withFocus }, { count: closed }
  ] = await Promise.all([
    supabase.from('properties').select('*', { count: 'exact', head: true
}).eq('active', true),
    supabase.from('visits').select('*', { count: 'exact', head: true
}).gte('visited_at', today),
    supabase.from('visits').select('*', { count: 'exact', head: true })
      .eq('status', 'visitado_com_achado').gte('visited_at', today),
    supabase.from('visits').select('*', { count: 'exact', head: true })
      .eq('status', 'fechado').gte('visited_at', today),
  ])

  const coverage = total && visitedToday ? Math.round((visitedToday / total) * 100) : 0
  return { total: total ?? 0, visitedToday: visitedToday ?? 0,
           withFocus: withFocus ?? 0, closed: closed ?? 0, coverage }
}

export default async function DashboardPage() {
  const m = await getMetrics()
  return (
    <div className="p-6 space-y-6">
      <div>
        <h1 className="text-2xl font-bold text-white">Dashboard</h1>
        <p className="text-slate-400 text-sm mt-1">Visao geral de hoje</p>
      </div>
      <div className="grid grid-cols-2 md:grid-cols-4 gap-4">
        <MetricCard label="Imoveis totais" value={m.total.toString()} color="blue" />
        <MetricCard label="Cobertura hoje" value={`${m.coverage}%`} color="green" />
        <MetricCard label="Com achado" value={m.withFocus.toString()} color="red" />
        <MetricCard label="Fechados" value={m.closed.toString()} color="gray" />
      </div>
      <div className="flex gap-3">
        <a href="/dashboard/imoveis"
          className="px-4 py-2 bg-blue-600 text-white rounded-lg text-sm font-semibold
hover:bg-blue-700">
          Ver imóveis
        </a>
        <a href="/api/export?days=30"
          className="px-4 py-2 bg-slate-700 text-white rounded-lg text-sm font-semibold
```

```
      </div>
    </div>
  )
}
```

web/components/MetricCard.tsx

```
interface Props {
  label: string; value: string
  color: 'blue' | 'green' | 'red' | 'gray' | 'yellow'
}

const colors = {
  blue:   'text-blue-400  border-blue-500/30  bg-blue-500/10',
  green:  'text-green-400 border-green-500/30 bg-green-500/10',
  red:    'text-red-400   border-red-500/30   bg-red-500/10',
  yellow: 'text-yellow-400 border-yellow-500/30 bg-yellow-500/10',
  gray:   'text-slate-400 border-slate-600 bg-slate-800/50',
}

export default function MetricCard({ label, value, color }: Props) {
  return (
    <div className={`rounded-xl border p-4 ${colors[color]}`}>
      <div className="text-3xl font-extrabold mb-1">{value}</div>
      <div className="text-xs uppercase tracking-wide opacity-70">{label}</div>
    </div>
  )
}
```

web/app/api/export/route.ts

```
import { NextRequest, NextResponse } from 'next/server'
import { createClient } from '@/lib/supabase'

export async function GET(req: NextRequest) {
  const supabase = createClient()
  const days = req.nextUrl.searchParams.get('days') ?? '30'
  const since = new Date()
  since.setDate(since.getDate() - parseInt(days))
```

```
  const { data, error } = await supabase
    .select('visited_at, status, notes, properties(code, address, type),
profiles(name)')
    .gte('visited_at', since.toISOString())
    .order('visited_at', { ascending: false })

  if (error) return NextResponse.json({ error: error.message }, { status: 500 })

  const header = 'Data,Hora,Codigo,Endereco,Tipo,Status,Agente,Nota\n'
  const rows = (data ?? []).map((v: any) => {
    const d = new Date(v.visited_at)
    return [
      d.toLocaleDateString('pt-BR'),
      d.toLocaleTimeString('pt-BR', { hour: '2-digit', minute: '2-digit' }),
      v.properties?.code ?? '',
      `"${v.properties?.address ?? ''}"`,
      v.properties?.type ?? '', v.status,
      v.profiles?.name ?? '',
      `"${(v.notes ?? '').replace(/"/g, '""')}"`,
    ].join(',')
  }).join('\n')

  return new NextResponse(header + rows, {
    headers: {
      "Content-Type": 'text/csv; charset=utf-8',
      "Content-Disposition": `attachment; filename="visitas_${days}dias.csv"`,
    },
  })
}
```

5 Configurações e Deploy

Variáveis de ambiente, app.json, eas.json e checklist de setup.

📄 app/.env

```
EXPO_PUBLIC_SUPABASE_URL=https://SEU_PROJETO.supabase.co
EXPO_PUBLIC_SUPABASE_ANON_KEY=eyJ...sua_anon_key
```

ZoonoseApp Scaffold v1.0

Expo + Supabase + Next.js · Android-first MVP

```
{
  "expo": {
    "name": "ZoonoseApp",
    "slug": "zoonoseapp",
    "version": "1.0.0",
    "orientation": "portrait",
    "icon": "./assets/icon.png",
    "splash": { "backgroundColor": "#0a1628" },
    "android": {
      "adaptiveIcon": { "backgroundColor": "#0a1628" },
      "package": "br.com.zoonoseapp"
    },
    "plugins": [
      ["expo-image-picker", { "photosPermission": "Necessario para fotos de evidencia."
}],
      ["expo-camera", { "cameraPermission": "Necessario para fotografar evidencias." }]
    ]
  }
}
```

app/eas.json

```
{
  "cli": { "version": ">= 12.0.0" },
  "build": {
    "development": {
      "developmentClient": true,
      "distribution": "internal"
    },
    "preview": {
      "android": { "buildType": "apk", "distribution": "internal" }
    },
    "production": {
      "android": { "buildType": "aab" }
    }
  }
}
```

web/.env.local

```
SUPABASE_SERVICE_ROLE_KEY=eyJ...service_role_key
```

CHECKLIST DE SETUP

1. Criar conta no Supabase (supabase.com) — plano Free
2. Criar novo projeto → copiar URL e anon key
3. Abrir SQL Editor → colar e rodar `001_schema.sql`
4. Rodar `002_indexes.sql` se houver
5. Criar usuários no Supabase Auth → Authentication → Users
6. Rodar `003_seed.sql` (setores e imóveis de demo)
7. No app/: copiar `.env.example` → `.env` → preencher URL e key
8. `npm install && npx expo start`
9. Testar login no Expo Go (Android)
10. No web/: copiar `.env.example` → `.env.local` → preencher
11. `npm install && npm run dev` → abrir localhost:3000
12. Para gerar APK: `eas login` → `eas build --platform android --profile preview`

6 Import de Imóveis via CSV

Script Node.js para importar planilha. Formato: `codigo,endereco,setor_nome,tipo`

ZoonoseApp Scaffold v1.0 Expo + Supabase + Next.js · Android-first MVP

```
// USO: node import-imoveis.js imoveis.csv
// Formato: codigo,endereco,setor_nome,tipo
// Requer: npm install @supabase/supabase-js csv-parse dotenv

require('dotenv').config({ path: '../app/.env' })
const { createClient } = require('@supabase/supabase-js')
const { parse } = require('csv-parse/sync')
const fs = require('fs')

const supabase = createClient(
  process.env.EXPO_PUBLIC_SUPABASE_URL,
  process.env.EXPO_PUBLIC_SUPABASE_ANON_KEY
)

async function main() {
  const file = process.argv[2]
  if (!file) { console.error('Informe o arquivo CSV'); process.exit(1) }

  const content = fs.readFileSync(file, 'utf8')
  const records = parse(content, { columns: true, skip_empty_lines: true })

  const { data: sectors } = await supabase.from('sectors').select('id, name')
  const sectorMap = Object.fromEntries(sectors.map(s => [s.name.toLowerCase(), s.id]))

  const properties = records
    .map(r => ({
      code:      r.codigo?.trim(),
      address:   r.endereco?.trim(),
      sector_id: sectorMap[r.setor_nome?.trim().toLowerCase()],
      type:      r.tipo?.trim() || 'residencial',
    }))
    .filter(p => p.code && p.address)

  console.log(`Importando ${properties.length} imoveis...`)

  const { error } = await supabase
    .from('properties')
    .upsert(properties, { onConflict: 'code' })

  if (error) { console.error('Erro:', error.message); process.exit(1) }
  console.log(`OK: ${properties.length} imoveis importados com sucesso.`)
}
```

ZoonoseApp Scaffold v1.0 Expo + Supabase + Next.js · Android-first MVP

EXEMPLO DE CSV

imoveis.csv

```
codigo,endereco,setor_nome,tipo
UBL-0001,Rua das Acacias 123 Tibery,Setor Norte,residencial
UBL-0002,Av Joao Naves de Avila 456 Santa Monica,Setor Norte,residencial
UBL-0003,Rua Olegario Maciel 78 Centro,Setor Sul,comercial
```

app/src/screens/PropertyDetailScreen.tsx

```
import React, { useEffect, useState } from 'react'
import { View, Text, StyleSheet, TouchableOpacity, ActivityIndicator, ScrollView } from
'react-native'
import { useRoute, useNavigation, RouteProp } from '@react-navigation/native'
import { supabase } from '../lib/supabase'
import { Property, Visit, STATUS_CONFIG } from '../types'
import { RootStackParamList } from '../navigation/AppNavigator'

type RouteType = RouteProp<RootStackParamList, 'PropertyDetail'>

export default function PropertyDetailScreen() {
  const route = useRoute<RouteType>()
  const nav = useNavigation()
  const [property, setProperty] = useState<Property | null>(null)
  const [lastVisit, setLastVisit] = useState<Visit | null>(null)
  const [loading, setLoading] = useState(true)

  useEffect(() => {
    async function load() {
      const [{ data: prop }, { data: visits }] = await Promise.all([
        supabase.from('properties').select('*').eq('id',
route.params.propertyId).single(),
        supabase.from('visits').select('*, profiles(name)')
          .eq('property_id', route.params.propertyId)
          .order('visited_at', { ascending: false }).limit(1),
      ])
```

ZoonoseApp Scaffold v1.0 Expo + Supabase + Next.js · Android-first MVP

```
    }
    load()
  }, [route.params.propertyId])

  if (loading) return <ActivityIndicator color="#3b82f6" style={{ flex: 1 }} />
  if (!property) return null

  const statusCfg = lastVisit ? STATUS_CONFIG[lastVisit.status] : null

  return (
    <ScrollView style={s.container} contentContainerStyle={{ padding: 20 }}>
      <View style={s.card}>
        <Text style={s.codeLabel}>Código</Text>
        <Text style={s.code}>{property.code}</Text>
        <Text style={s.addressLabel}>Endereço</Text>
        <Text style={s.address}>{property.address}</Text>
        <View style={s.row}>
          <View style={s.chip}><Text style={s.chipText}>{property.type}</Text></View>
          {statusCfg && (
            <View style={[s.chip, { borderColor: statusCfg.color, backgroundColor:
statusCfg.bg }]}>
              <Text style={[s.chipText, { color: statusCfg.color }]}>{statusCfg.label}
</Text>
            </View>
          )}
        </View>
      </View>

      {lastVisit && (
        <View style={s.card}>
          <Text style={s.sectionLabel}>Última visita</Text>
          <Text style={s.visitDate}>
            {new Date(lastVisit.visited_at).toLocaleDateString('pt-BR')}
          </Text>
          {lastVisit.notes && <Text style={s.notes}>{lastVisit.notes}</Text>}
        </View>
      )}

      <TouchableOpacity style={s.btn}
        onPress={() => nav.navigate('RegisterVisit' as any, {
          propertyId: property.id, propertyCode: property.code
        })}>
        <Text style={s.btnText}>Registrar nova visita</Text>
```

ZoonoseApp Scaffold v1.0 Expo + Supabase + Next.js · Android-first MVP

```
}

const s = StyleSheet.create({
  container:    { flex: 1, backgroundColor: '#0a1628' },
  card:         { backgroundColor: '#111d35', borderRadius: 12, padding: 16,
marginBottom: 16,
                  borderWidth: 1, borderColor: '#1e3a5f' },
  codeLabel:    { color: '#94a3b8', fontSize: 10, fontWeight: '700', textTransform:
'uppercase', letterSpacing: 1, marginBottom: 4 },
  code:         { color: '#60a5fa', fontWeight: '700', fontSize: 20, fontFamily:
'monospace', marginBottom: 12 },
  addressLabel: { color: '#94a3b8', fontSize: 10, fontWeight: '700', textTransform:
'uppercase', letterSpacing: 1, marginBottom: 4 },
  address:      { color: '#e2e8f0', fontSize: 14, marginBottom: 12 },
  row:          { flexDirection: 'row', gap: 8 },
  chip:         { borderWidth: 1, borderColor: '#1e3a5f', borderRadius: 6,
paddingHorizontal: 10, paddingVertical: 4 },
  chipText:     { color: '#94a3b8', fontSize: 12, fontWeight: '600' },
  sectionLabel: { color: '#94a3b8', fontSize: 10, fontWeight: '700', textTransform:
'uppercase', letterSpacing: 1, marginBottom: 8 },
  visitDate:    { color: '#e2e8f0', fontSize: 15, fontWeight: '600' },
  notes:        { color: '#94a3b8', fontSize: 13, marginTop: 6 },
  btn:          { backgroundColor: '#3b82f6', borderRadius: 10, padding: 16,
alignItems: 'center', marginTop: 8 },
  btnText:      { color: '#fff', fontWeight: '700', fontSize: 15 },
})
```

app/src/screens/ConfirmVisitScreen.tsx

```
import React from 'react'
import { View, Text, TouchableOpacity, StyleSheet } from 'react-native'
import { useRoute, useNavigation } from '@react-navigation/native'
import { STATUS_CONFIG } from '../types'

export default function ConfirmVisitScreen() {
  const route = useRoute() as any
  const nav = useNavigation()
  const { visitData } = route.params
  const cfg = STATUS_CONFIG[visitData.status as keyof typeof STATUS_CONFIG]

  return (
```

```
      {visitData.offline && (
        <View style={s.offlineBadge}>
          <Text style={s.offlineText}>Salvo offline — será sincronizado
automaticamente</Text>
        </View>
      )}
      <View style={[s.statusCard, { borderColor: cfg?.color }]}>
        <Text style={[s.statusText, { color: cfg?.color }]}>{cfg?.label ??
visitData.status}</Text>
      </View>
      <Text style={s.date}>
        {new Date(visitData.visited_at).toLocaleString('pt-BR')}
      </Text>
      <TouchableOpacity style={s.btn} onPress={() => nav.navigate('Home' as any)}>
        <Text style={s.btnText}>Voltar ao início</Text>
      </TouchableOpacity>
    </View>
  )
}

const s = StyleSheet.create({
  container:    { flex: 1, backgroundColor: '#0a1628', alignItems: 'center',
justifyContent: 'center', padding: 32 },
  icon:         { width: 72, height: 72, borderRadius: 36, backgroundColor: '#10b981',
                  alignItems: 'center', justifyContent: 'center', marginBottom: 20 },
  iconText:     { color: '#fff', fontSize: 32, fontWeight: '800' },
  title:        { color: '#e2e8f0', fontSize: 22, fontWeight: '700', marginBottom: 20
},
  offlineBadge: { backgroundColor: 'rgba(59,130,246,0.15)', borderWidth: 1,
borderColor: '#3b82f6',
                  borderRadius: 8, padding: 10, marginBottom: 16 },
  offlineText:  { color: '#60a5fa', fontSize: 13, textAlign: 'center' },
  statusCard:   { borderWidth: 2, borderRadius: 12, paddingHorizontal: 24,
paddingVertical: 12, marginBottom: 16 },
  statusText:   { fontWeight: '700', fontSize: 16 },
  date:         { color: '#94a3b8', fontSize: 13, marginBottom: 32 },
  btn:          { width: '100%', backgroundColor: '#3b82f6', borderRadius: 10, padding:
16, alignItems: 'center' },
  btnText:      { color: '#fff', fontWeight: '700', fontSize: 15 },
})
```

ZoonoseApp Scaffold v1.0 Expo + Supabase + Next.js · Android-first MVP

app/src/screens/HistoryScreen.tsx

```
import React, { useEffect, useState } from 'react'
import { View, Text, FlatList, StyleSheet, ActivityIndicator } from 'react-native'
import { supabase } from '../lib/supabase'
import { useAuth } from '../hooks/useAuth'
import { Visit, STATUS_CONFIG } from '../types'

export default function HistoryScreen() {
  const { profile } = useAuth()
  const [visits, setVisits] = useState<Visit[]>([])
  const [loading, setLoading] = useState(true)

  useEffect(() => {
    if (!profile) return
    supabase
      .from('visits')
      .select('*, properties(code, address)')
      .eq('agent_id', profile.id)
      .order('visited_at', { ascending: false })
      .limit(50)
      .then(({ data }) => { setVisits(data ?? []); setLoading(false) })
  }, [profile])

  if (loading) return <ActivityIndicator color="#3b82f6" style={{ flex: 1 }} />

  return (
    <FlatList
      style={{ flex: 1, backgroundColor: '#0a1628' }}
      contentContainerStyle={{ padding: 16 }}
      data={visits}
      keyExtractor={v => v.id}
      ListEmptyComponent={<Text style={{ color: '#94a3b8', textAlign: 'center',
marginTop: 40 }}>Nenhuma visita registrada.</Text>}
      renderItem={({ item }) => {
        const cfg = STATUS_CONFIG[item.status]
        const prop = (item as any).properties
        return (
          <View style={s.item}>
            <View style={[s.statusDot, { backgroundColor: cfg.color }]} />
            <View style={{ flex: 1 }}>
              <Text style={s.code}>{prop?.code ?? item.property_id.slice(0,8)}</Text>
              <Text style={s.address} numberOfLines={1}>{prop?.address ?? '-'}</Text>
            </View>
            <View>
```

ZoonoseApp Scaffold v1.0 Expo + Supabase + Next.js · Android-first MVP

```
              </Text>
            </View>
          </View>
        )
      }}
    />
  )
}

const s = StyleSheet.create({
  item:      { flexDirection: 'row', alignItems: 'center', backgroundColor: '#111d35',
               borderRadius: 10, padding: 14, marginBottom: 8, gap: 12,
               borderWidth: 1, borderColor: '#1e3a5f' },
  statusDot: { width: 10, height: 10, borderRadius: 5, flexShrink: 0 },
  code:      { color: '#60a5fa', fontWeight: '700', fontSize: 12, marginBottom: 2 },
  address:   { color: '#e2e8f0', fontSize: 13 },
  status:    { fontSize: 11, fontWeight: '700', textAlign: 'right', marginBottom: 2 },
  date:      { color: '#94a3b8', fontSize: 11, textAlign: 'right' },
})
```

app/src/screens/SyncStatusScreen.tsx

```
import React, { useState } from 'react'
import { View, Text, TouchableOpacity, FlatList, StyleSheet } from 'react-native'
import { offlineQueue } from '../lib/mmkv'
import { useOfflineSync } from '../hooks/useOfflineSync'

export default function SyncStatusScreen() {
  const [queue, setQueue] = useState(offlineQueue.getAll())
  const { syncPending } = useOfflineSync()

  const handleSync = async () => {
    await syncPending()
    setQueue(offlineQueue.getAll())
  }

  return (
    <View style={s.container}>
      <View style={s.header}>
        <Text style={s.title}>{queue.length} pendente(s)</Text>
        <TouchableOpacity style={s.syncBtn} onPress={handleSync}>
```

ZoonoseApp Scaffold v1.0 Expo + Supabase + Next.js · Android-first MVP

```
      <FlatList
        data={queue}
        keyExtractor={i => i.localId}
        ListEmptyComponent={
          <Text style={s.empty}>Nenhum dado pendente de sincronização.</Text>
        }
        renderItem={({ item }) => (
          <View style={s.item}>
            <Text style={s.itemId}>{item.property_id.slice(0,8)}</Text>
            <Text style={s.itemStatus}>{item.status}</Text>
            <Text style={s.itemDate}>
              {new Date(item.visited_at).toLocaleDateString('pt-BR')}
            </Text>
          </View>
        )}
      />
    </View>
  )
}

const s = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#0a1628', padding: 16 },
  header:    { flexDirection: 'row', justifyContent: 'space-between', alignItems:
'center', marginBottom: 16 },
  title:     { color: '#e2e8f0', fontWeight: '700', fontSize: 16 },
  syncBtn:   { backgroundColor: '#3b82f6', borderRadius: 8, paddingHorizontal: 16,
paddingVertical: 8 },
  syncBtnText: { color: '#fff', fontWeight: '700', fontSize: 13 },
  empty:     { color: '#94a3b8', textAlign: 'center', marginTop: 40 },
  item:      { flexDirection: 'row', justifyContent: 'space-between', backgroundColor:
'#111d35',
               borderRadius: 10, padding: 14, marginBottom: 8, borderWidth: 1,
borderColor: '#1e3a5f' },
  itemId:    { color: '#60a5fa', fontFamily: 'monospace', fontSize: 12 },
  itemStatus: { color: '#e2e8f0', fontSize: 12 },
  itemDate:  { color: '#94a3b8', fontSize: 12 },
})
```

📄 app/src/screens/SettingsScreen.tsx

ZoonoseApp Scaffold v1.0 Expo + Supabase + Next.js · Android-first MVP

```
import { View, Text, TouchableOpacity, StyleSheet, Alert } from 'react-native'
import { useAuth } from '../hooks/useAuth'
import { offlineQueue } from '../lib/mmkv'

export default function SettingsScreen() {
  const { profile, signOut } = useAuth()

  const handleClearQueue = () =>
    Alert.alert('Limpar fila', 'Excluir visitas offline nao sincronizadas?', [
      { text: 'Cancelar', style: 'cancel' },
      { text: 'Excluir', style: 'destructive',
        onPress: () => offlineQueue.clear() },
    ])

  return (
    <View style={s.container}>
      <View style={s.profile}>
        <View style={s.avatar}><Text style={s.avatarText}>{profile?.name?.
[0]?.toUpperCase()}</Text></View>
        <View>
          <Text style={s.name}>{profile?.name}</Text>
          <Text style={s.role}>{profile?.role}</Text>
        </View>
      </View>

      <TouchableOpacity style={s.item} onPress={handleClearQueue}>
        <Text style={s.itemText}>Limpar fila offline</Text>
        <Text style={s.arrow}>›</Text>
      </TouchableOpacity>

      <TouchableOpacity style={[s.item, s.danger]} onPress={signOut}>
        <Text style={[s.itemText, { color: '#ef4444' }]}>Sair da conta</Text>
        <Text style={[s.arrow, { color: '#ef4444' }]}>›</Text>
      </TouchableOpacity>
    </View>
  )
}

const s = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#0a1628', padding: 20 },
  profile:   { flexDirection: 'row', alignItems: 'center', gap: 14, backgroundColor:
'#111d35',
               borderRadius: 12, padding: 16, marginBottom: 24, borderWidth: 1,
borderColor: '#1e3a5f' },
```

ZoonoseApp Scaffold v1.0 Expo + Supabase + Next.js · Android-first MVP

```
  name:      { color: '#e2e8f0', fontWeight: '700', fontSize: 15 },
  role:      { color: '#94a3b8', fontSize: 12, marginTop: 2 },
  item:      { flexDirection: 'row', justifyContent: 'space-between', alignItems:
'center',
               backgroundColor: '#111d35', borderRadius: 10, padding: 16, marginBottom:
8,
               borderWidth: 1, borderColor: '#1e3a5f' },
  itemText:  { color: '#e2e8f0', fontSize: 14 },
  arrow:     { color: '#94a3b8', fontSize: 18 },
  danger:    { borderColor: 'rgba(239,68,68,0.3)' },
})
```

ZoonoseApp — Scaffold MVP v1.0 — Março 2026 — Stack: Expo + Supabase + Next.js